Assignment 2: Multivariate Statistics 2025/2026

Group 14

05-01-2026

Michele Garbin (1086449)

Arad Mehralian (1074638)

Athulya Mohandas Menon (1070436)

Lucas Vlasblom (1086552)

Task 1

Question A

Data dimensionality reduction

As per the question, we load the data (split into train and validation and 2 scenarios with one having less training data than the other) for both scenarios. Next, we perform a PCA on the centred data using `prcomp`. We can avoid scaling here since the data involves handwritten samples of 4 letters, and hence, we can assume they are not on vastly differing scales, which is when standardising the data would be required.

```
pca_s1 <- prcomp(train.data.s1, center = T, scale. = F)
```

After running PCA on the train data for both scenarios, we then compute the cumulative variance to pick the number of components for which the variance explained exceeds 90%.

```
var_prop_s1 <- cumsum(pca_s1$sdev^2) / sum(pca_s1$sdev^2)
num_comp_s1 <- which(var_prop_s1 >= 0.9)[1]
```

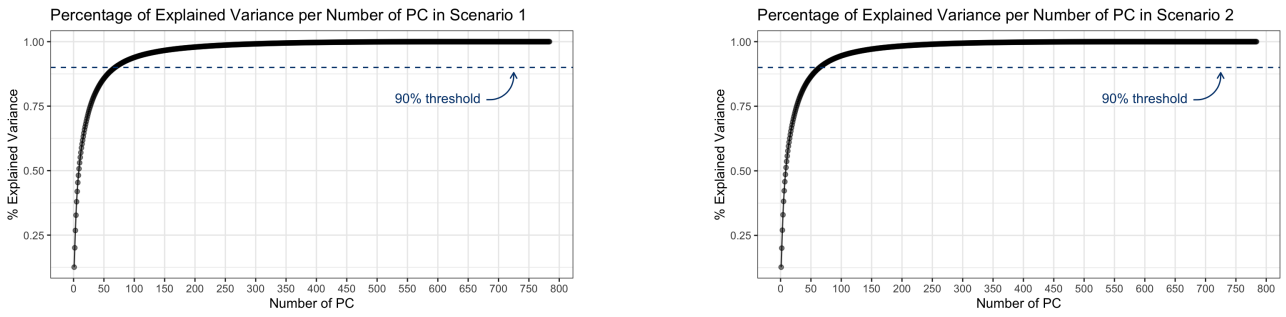

Figure 1: Scree plot for PCA on both Scenarios

Through this process, we find that for Scenario 1, 69 PCs can explain over 90% of variance, while for Scenario 2, only 65 PCs are required to convey more than 90% variance. We notice that there is a small difference between the number of components needed to sufficiently explain the two datasets. The larger dataset needs four more components to explain the same amount of variance. This could be due to a larger variability within the first dataset.

Next, we rotate the training data for both scenarios into a new coordinate space where the variables are essentially uncorrelated using only the PCs we selected above.

```
train_pc_s1 <- pca_s1$x [, 1:num_comp_s1]
train_pc_s2 <- pca_s2$x [, 1:num_comp_s2]
test_pc_s1 <- scale (test.data, center = pca_s1$center, scale = F)
                    %*% pca_s1$rotation [, 1:num_comp_s1]
test_pc_s2 <- scale (test.data, center = pca_s2$center, scale = F)
                    %*% pca_s2$rotation [, 1:num_comp_s2]
```

The test data has to be projected onto the same coordinate system as the training data. We centre and rotate the test set using the parameters from the training set. This ensures that our model creates predictions based on the same space, preventing bias and retaining representativity.

Wilk's Lambda Tests

We then check our assumption that the data is separable into different classes using the Wilk's Lambda test for both scenarios.

The very small Λ value (0.03) suggests that the data are well-separated within the PC space, and the high $\mathcal{F}$ value ($\sim$274) highlights that the between-group differences are higher than the within-group ones, indicating that proceeding with discriminant analysis is warranted in this case. The minimal p-value also suggests that the separation between classes is statistically significant.

```
wilk_s1 <- manova(as.matrix(train_pc_s1) ~ train.target.s1)
summary(wilk_s1, test = "Wilks")
                  Df   Wilks approx F num Df den Df    Pr(>F)
train.target.s1    3 0.03766   273.97    207  28572 < 2.2e-16 ***
Residuals       9596
```

Here again, we see a very small Λ value (0.03), suggesting strong class-separation within the data. Even though the F value ($\sim$52) is lower than for Scenario 1, it is still high enough to indicate that the groups do differ strongly enough to attempt discriminant analysis. Also, since this dataset has far fewer data points than the first one, the lower F value may be a symptom of that. In this case as well, the p-value is small enough to say that group differences are statistically significant.

```
wilk_s2 <- manova(as.matrix(train_pc_s2) ~ train.target.s2)
summary(wilk_s2, test = "Wilks")
                  Df    Wilks approx F num Df den Df    Pr(>F)
train.target.s2    3 0.030713   51.721    195 4594.5 < 2.2e-16 ***
Residuals       1596
```

Question B

Linear Discriminant Analysis

We now perform LDA on both Scenario 1 and Scenario 2 by passing to the function the data that has been transposed onto the PC dimensions.

```
lda_s1 <- lda(train_pc_s1, grouping = train.target.s1)
lda_train_pred_s1 <- predict(lda_s1)$class
Prior probabilities of groups:
   D    G    O    Q
0.25 0.25 0.25 0.25
```

From prior probabilities, we can see that we have a balanced dataset, which is ideal as it prevents any bias that may be possible towards the larger classes. The group means within the PCA space show us that each class has different values, indicating their separability within the PCA dimension.

```
Proportion of trace:
   LD1    LD2    LD3
0.5817 0.2295 0.1888
```

We can see that LD1 explains most of the between-group separation, with LD2 and LD3 providing smaller improvements.

```
table(lda_train_pred_s1, train.target.s1)
                 train.target.s1
lda_train_pred_s1    D    G    O    Q
                D 2138   25   94   62
                G   23 2125   21  108
                O  199   52 2265   78
                Q   40  198   20 2152
```

From the confusion matrix, we can see that LDA does a decent job, with most of the errors coming from visually similar classes – D and O and G and Q.

The posterior probability table shows that overall LDA predicts confidently, and that lower confidence is linked to the visually similar classes, which also makes sense intuitively.

```
lda_train_err_s1 <- mean(lda_train_pred_s1 != train.target.s1)
lda_test_pred_s1 <- predict(lda_s1, newdata = test_pc_s1)$class
lda_test_err_s1 <- mean(lda_test_pred_s1 != test.target)
```

The train and test errors (9% and 11%) are not dissimilar enough to feel there is heavy overfitting, and overall, the LDA does a decent job. G and Q, D and O are most confused with each other.

For Scenario 2 as well, LD1 explains most of the separation between classes, and misclassifications are most common between visually similar classes. The train and test error differences are higher here (8.6% and 12.7%), indicating slight overfitting. The fewer samples in Scenario 2, when compared to Scenario 1, may be the cause for less stable parameters and higher variance.

The `boxM()` test for homoscedasticity across the four classes rejects H_0 in both Scenarios. This is crucial, as it tells us the assumption behind LDA is not valid, and we should opt for something more flexible, like QDA, for instance. However, we should interpret this outcome cautiously. Since the test relies on the assumption of multivariate normality, which is violated here,[1] the rejection of H_0 may be spurious. In such cases, the test tends to confound non-normality with variance inequality.

Quadratic Discriminant Analysis

```
qda_s1 <- qda(train_pc_s1, grouping = train.target.s1)
qda_train_pred_s1 <- predict(qda_s1)$class
table(qda_train_pred_s1, train.target.s1)
qda_train_err_s1 <- mean(qda_train_pred_s1 != train.target.s1)
qda_test_pred_s1 <- predict(qda_s1, newdata = test_pc_s1)$class
qda_test_err_s1 <- mean(qda_test_pred_s1 != test.target)
```

In Scenario 1, the train error is 4.6%, and the test error is 5.8%, whereas for Scenario 2, the train error is 2.6% and the test error is 6.8%. The confusion matrix shows that there are fewer errors here than in LDA, probably due to the curved decision boundary, which allows for better separation. As in the LDA, confusion still exists between classes that look similar (D and O & G and Q), but the train and test errors are much lower, indicating that the QDA performs better.

As with LDA, the test error for scenario 2 is much bigger than its train error, suggesting the smaller sample might have less capability of generalising the results.

[1] To check for normality, we looked at 65 + 69 QQ-plot and, for both Scenarios, almost 50% of the PC show significant deviation from normality.

K-Nearest Neighbours

```
k_vals <- seq(1, 20);num_k <- length(k_vals)
train_err <- numeric(num_k); test_err <- numeric(num_k)
for (i in seq_along(k_vals))
{
k <- k_vals[i]
train_pred <- knn(train = train_pc_s1, test = train_pc_s1, cl = train.target.s1, k = k)
train_err[i] <- mean(train_pred != train.target.s1)
test_pred <- knn(train = train_pc_s1, test = test_pc_s1, cl = train.target.s1, k = k)
test_err[i] <- mean(test_pred != test.target)
}
best_k_val_s1 <- k_vals[which.min(test_err)]
```

We test k values from 1 to 20, to find the best number of nearest neighbours which would give us the lowest error rate, to build the final model using that value. While $k = 1$ may give the best error, it does not make sense to use that since it would create an extremely flexible model that would overfit easily. Similarly, choosing a k too high could lead to a model averaging too many values, which might result in underfitting. In this case, for both scenarios, the best k-value was found to be 3 since it smoothened out noise while also separating the classes effectively.

```
train_pred_knn_s1 <- knn(train = train_pc_s1, test = train_pc_s1,
                         cl = train.target.s1, k = best_k_val_s1)
test_pred_knn_s1 <- knn(train = train_pc_s1, test = test_pc_s1,
                         cl = train.target.s1, k = best_k_val_s1)
knn_train_err_s1 <- mean(train_pred_knn_s1 != train.target.s1)
knn_test_err_s1 <- mean(test_pred_knn_s1 != test.target)
```

After fitting the models with $k = 3$, we can see that for Scenario 1, the train vs test error gap (4.1% vs 7.75%) was not too bad, indicating decent generalisation, whereas the gap for Scenario 2 (6.4% vs 12.75%) showed overfitting due to the smaller sample size.

Multinomial Regression

```
mul_s2 <- multinom( train.target.s2 ~ ., data = as.data.frame(train_pc_s2),
            family = multinomial, maxit = 1000, hess = TRUE )
train_pred_s2 <- predict(mul_s2, newdata = as.data.frame (train_pc_s2))
test_pred_s2 <- predict(mul_s2, newdata = as.data.frame( test_pc_s2))
```

The Multinomial Regression extends logistic regression to multiple classes, and the iterative values show us that the model was able to reduce negative log-likelihood from ~ 13000 to ~ 2300, indicating that the model was able to learn a significant amount from the data.

```
mlr_train_err_s2 <- mean(train_pred_s2 != train.target.s2)
mlr_test_err_s2 <- mean(test_pred_s2 != test.target)
```

For Scenario 1, the difference between train and test error is minimal (8% vs. 9.3%), whereas for Scenario 2, it is significant (5% vs. 12%) due to having fewer datapoints. The fact that it appears weaker than QDA and similar to LDA does not surprise us much, since both LDA and Multinomial Regression fit linear boundaries to the data.

Multinomial Regression with Squared Terms

```
train_pc_s1_sq <- as.data.frame(train_pc_s1)
test_pc_s1_sq <- as.data.frame(test_pc_s1)
sqr_train_s1 <- train_pc_s1_sq^2
```

```
colnames(sqr_train_s1) <- paste0(colnames(train_pc_s1_sq), "_sq")
sqr_test_s1 <- test_pc_s1_sq^2
colnames(sqr_test_s1) <- paste0(colnames(test_pc_s1_sq), "_sq")
train_pc_s1_sq <- cbind(train_pc_s1_sq, sqr_train_s1)
test_pc_s1_sq <- cbind(test_pc_s1_sq, sqr_test_s1)
```

To capture potential non-linear relationships, we now run the multinomial regression on a dataset that comprises the original PCs along with their squared terms.

```
mul_sq_s1 <- multinom( train.target.s1 ~ ., data = train_pc_s1_sq,
                family = multinomial, maxit = 1000, hess = T )
train_pred_sq_s1 <- predict(mul_sq_s1, newdata = train_pc_s1_sq)
test_pred_sq_s1 <- predict(mul_sq_s1, newdata = test_pc_s1_sq)
mlr_sq_train_err_s1 <- mean(train_pred_sq_s1 != train.target.s1)
mlr_sq_test_err_s1 <- mean(test_pred_sq_s1 != test.target)
```

```
Scenario 1: K-1 classes = 3  Vars = 139  Parameters = 417  Training samples = 9600
Scenario 2: K-1 classes = 3  Vars = 131  Parameters = 393  Training samples = 1600
```

Scenario	Train Error	Test Error
Scenario 1	0.08052083	0.092500
Scenario 2	0.05187500	0.118125
Scenario 1 (PC + PC^2)	0.05520833	0.083750
Scenario 2 (PC + PC^2)	0.00000000	0.164375

Table 1: Train/test error for both Scenarios using Multinomial Regression, with and without the quadratic term

For Scenario 1, there is a distinct drop in both training and test errors, further proving that the actual boundary is not linear between the classes (8% vs. 5%), whereas for Scenario 2, the train error drops to 0 while test error rises to 16%, indicating high overfitting. In conclusion, since Scenario 1 has more observations, adding more parameters in the form of squared terms does not harm performance. On the contrary, since Scenario 2 has only 1600 samples, having 393 parameters leads to severe overfitting.

XGBoost

```
y_train_s1 <- as.numeric(train.target.s1) - 1
y_train_s2 <- as.numeric(train.target.s2) - 1
y_test <- as.numeric(test.target) - 1
dtrain_s1 <- xgb.DMatrix(data = as.matrix(train.pc.s1), label = y_train_s1)
dtest_s1 <- xgb.DMatrix(data = as.matrix(test.pc.s1), label = y_test)
```

To run multiclass-SoftMax predictions we converted the labels to 0,1,2,3 since XGB requires it to be 0-indexed. We then converted the train and test datasets into DMatrix which is an optimised data structure that XBG uses to run the process.

```
xgb_s1 <- xgb.train(data = dtrain_s1, nrounds = 100, objective = "multi:softmax",
            num_class = length(unique(y_train_s1)), max_depth = 10,
            eta = 0.5, subsample = 1, colsample_bytree = 1)
pred_train_s1 <- predict(xgb_s1, dtrain_s1)
pred_test_s1 <- predict(xgb_s1, dtest_s1)
xgbun_trn_err_s1 <- mean(pred_train_s1 != y_train_s1)
xgbun_tst_err_s1 <- mean(pred_test_s1 != y_test)
```

We then ran a rudimentary model with stock values and compared the test and training errors; we then provided the same values for both scenarios despite them having different number of samples. While for Scenario 1 the untuned model gave a training and test error respectively of 0% and 6.5% (indicating overfitting), for Scenario 2 the model did much worse (0% vs. 11.5%), highlighting that we needed to tune the models differently to match the scenario.

```
set.seed(1000)
cv_tune_s1 <- xgb.cv(data = dtrain_s1, nrounds = 300, nfold = 5,
                     objective = "multi:softmax",
                     num_class = length(unique(y_train_s1)),
                     max_depth = 6, eta = 0.2, subsample = 0.8,
                     colsample_bytree = 1, verbose = 0,
                     early_stopping_rounds = 20)
best_nrounds_tune_s1 <- cv_tune_s1$best_iteration
```

We looked at the performance and the data to then make some choices regarding the hyperparameters for the two scenarios and came to the following conclusions:

- In order to prevent overfitting, reduce `depth` to 6 for Scenario 1 and to 4 for Scenario 2. Scenario 2 needs less depth as it more prone to overfitting.
- For both Scenarios, pull back `eta` from 0.5 to the more theoretically established value of 0.2. This change essentially adds trees more gradually.
- To prevent overfitting, set `subsample` to 0.8 in both Scenarios. In this way, we are adding subsamples to increase the randomness in the process.
- For Scenario 1, set `colsample_bytree` (which controls the number of features sampled for each tree) to 1, since it has more data and can handle more features. For Scenario 2, set that parameter to 0.9 to avoid overfitting.

Rather than arbitrarily choosing number of rounds we had ran a CV to choose the best rounds, and then trained the model with the new parameters.

```
xgb_tune_s1 <- xgb.train(data = dtrain_s1, nrounds = best_nrounds_tune_s1,
                         objective = "multi:softmax",
                         num_class = length(unique(y_train_s1)), max_depth = 6,
                         eta = 0.2, subsample = 0.8,
                         colsample_bytree = 1, verbose = 0)
pred_trn_tune_s1 <- predict(xgb_tune_s1, dtrain_s1)
pred_tst_tune_s1 <- predict(xgb_tune_s1, dtest_s1)
xgb_trn_err_s1 <- mean(pred_trn_tune_s1 != y_train_s1)
xgb_tst_err_s1 <- mean(pred_tst_tune_s1 != y_test)
```

Tuning helped test error drop from 6.5% to 6% and from 11.5% to 9%, respectively for Scenario 1 and 2.

HDDA

```
s1_train_centered <- scale(train.data.s1, center = T, scale = F)
train_means_s1 <- colMeans(train.data.s1)
test_centered_s1 <- scale(test.data, center = train_means_s1, scale = F)
```

Since we must run HDDA on raw data we centered the training data and the test data (centered using training data means) and then ran the $[a_{kj}b_kQ_kD]$ model as specified.

```
hdda_s1 <- hdda(data = s1_train_centered, cls = train.target.s1, model = "AKJBKQKD",
                d = "Cattell", threshold = 0.05, scaling = F)
train_pred_hdda_s1 <- predict(hdda_s1, s1_train_centered, cls = train.target.s1)
```

```
hdda_train_err_s1 <- mean(train_pred_hdda_s1$class != train.target.s1)
test_pred_hdda_s1 <- predict(hdda_s1, test_centered_s1, cls = test.target)
hdda_test_err_s1 <- mean(test_pred_hdda_s1$class != test.target)
```

For Scenario 1 the gap between training and test errors (6.6% vs. 8.8%) indicates satisfactory generalisation, whereas it appears bigger for Scenario 2 (3.8% vs. 8.6%), suggesting overfitting.

Question C

All the results were combined into a dataframe, which is displayed in Table 2.

```
model_names <- rep(c("LDA", "QDA", "KNN", "Multinomial LogReg",
                     "Multinomial LogReg (Squared)", "XGB Untuned",
                     "XGB Tuned", "HDDA"), each = 2)
train_errors <- round(c(lda_trn_err_s1, lda_trn_err_s2, qda_trn_err_s1,
                        qda_trn_err_s2, knn_trn_err_s1, knn_trn_err_s2,
                        mlr_trn_err_s1, mlr_trn_err_s2, mlr_sq_trn_err_s1,
                        mlr_sq_trn_err_s2, xgbun_trn_err_s1, xgbun_trn_err_s2,
                        xgb_trn_err_s1,xgb_trn_err_s2, hdda_trn_err_s1,
                        hdda_trn_err_s2), 3)
test_errors <- round(c(lda_tst_err_s1, lda_tst_err_s2, qda_tst_err_s1,
                        qda_tst_err_s2, knn_tst_err_s1, knn_tst_err_s2,
                        mlr_tst_err_s1, mlr_tst_err_s2, mlr_sq_tst_err_s1,
                        mlr_sq_tst_err_s2, xgbun_tst_err_s1, xgbun_tst_err_s2,
                        xgb_tst_err_s1,xgb_tst_err_s2, hdda_tst_err_s1,
                        hdda_tst_err_s2), 3)
results <- data.frame(Model = model_names, Scenario = rep(c("Scenario 1", "Scenario 2"),
                      each = 1), Train_Error = train_errors, Test_Error = test_errors)
```

Table 2: Comparison of Training and Test Errors across different Scenarios and Models

	Scenario 1		Scenario 2	
Model	**Train**	**Test**	**Train**	**Test**
LDA	0.096	0.109	0.086	0.127
QDA	0.046	0.058	0.026	0.068
KNN	0.041	0.076	0.061	0.128
Multinomial LogReg	0.081	0.092	0.052	0.118
Multinomial LogReg (Squared)	0.055	0.084	0.000	0.168
Gradient Boosting	0.001	0.078	0.001	0.112
HDDA	0.067	0.083	0.039	0.086

For Scenario 1, the best performing model is the QDA, followed by the tuned XGB, which makes sense since the data exhibit a non-linear boundary (which is captured well by the QDA).

For Scenario 2, the best performing model is the QDA, followed by the HDDA and then the tuned XGB, This does not surprise us at all, since this set is more prone to overfitting when working with tree-based models due to fewer samples.

For ease of comparison, train and test error rates for each Scenario and displayed in Figure 2.

```
results_long <-results %>%
    pivot_longer(
        cols = c("Train_Error", "Test_Error"),
        names_to = "Error_Type",
```

```
            values_to = "Error_Value"
        )
results_long$Error_Type <- factor(results_long$Error_Type,
                                  levels = c("Train_Error", "Test_Error"))

ggplot(results_long, aes(x = Model, y = Error_Value, fill = Scenario)) +
  geom_bar(stat="identity", position = "dodge") +
  facet_wrap(~Error_Type) +
  labs(
    title = "Overview of model performance",
    y = "Error Rate",
    x = "Method"
  ) +
  theme_bw()+
  theme(axis.text.x = element_text(hjust = 1, angle = 45),
        legend.position = "top") +
  scale_fill_manual(name = "", values = c("#52BDEC", "#00407A"))
```

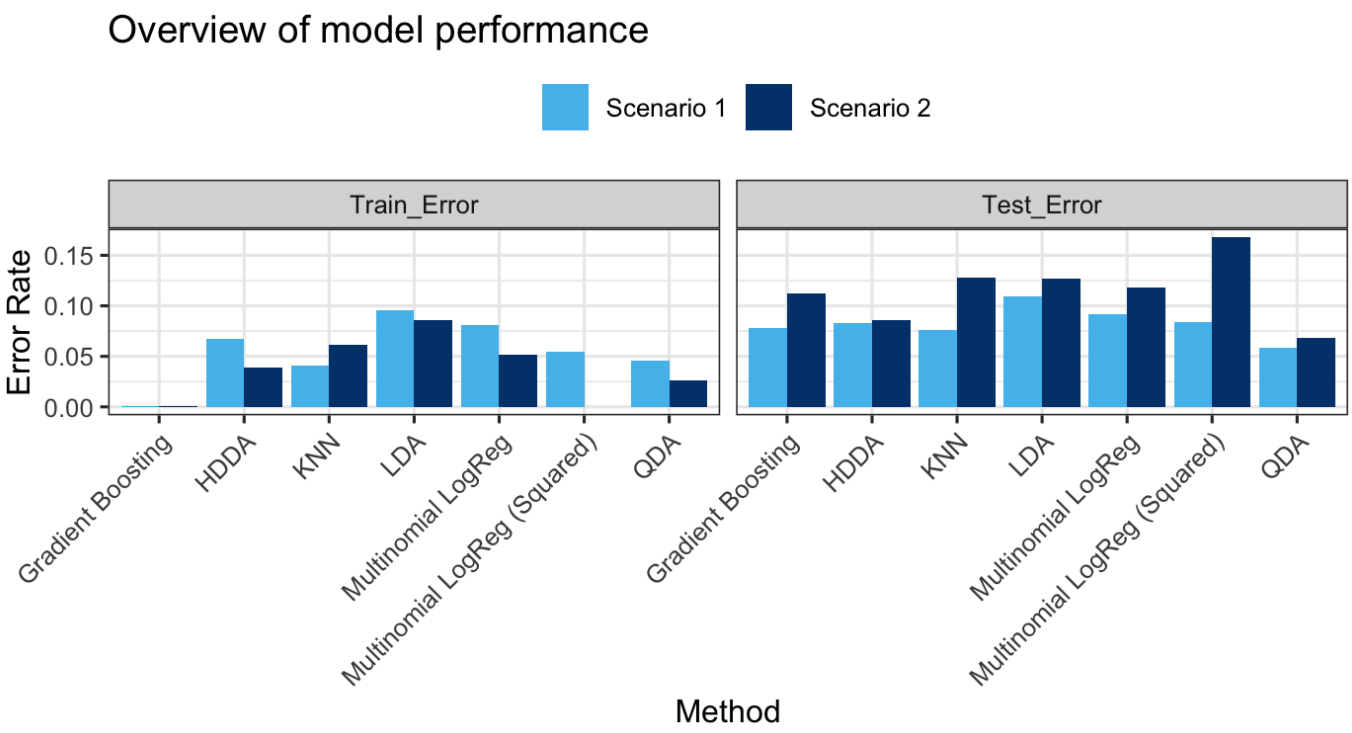

Figure 2: Train and Test Error Rates for different Models and Scenarios

Task 2

Question A

Data Preparation

```
load("beer.Rdata")
beer_mat <- as.matrix(beer)
dim(beer_mat)
sum(is.na(beer_mat))
```

We first load the data into R and then convert it into a numerical data matrix, since that is the most efficient way to calculate distances. We also check the dimensions of the data (399, 12) and how many `NULL` values are there (0).

Hierarchical Clustering - Ward's Method

```
dist_beer <- dist(t(beer_mat), method = "euclidean")^2
hc_beer <- hclust(dist_beer, method = "ward.D2")
plot(hc_beer, main = "Dendrogram Ward's Method")
rect.hclust(hc_beer, k = 3, border = 2:4)
```

To proceed with hierarchical clustering, we first transpose the matrix so that the clustering is done on the beers themselves, rather than those people who rated them. In this way, each beer appears as a row with 399 features. Next, pairwise Euclidean distances are calculated between beers. The resulting value squared is passed to the `hclust` function, to perform an agglomerative hierarchical clustering. In Figure 3 we plot the dendogram plus 3 coloured rectangles, which indicate the 3 distinct groups.

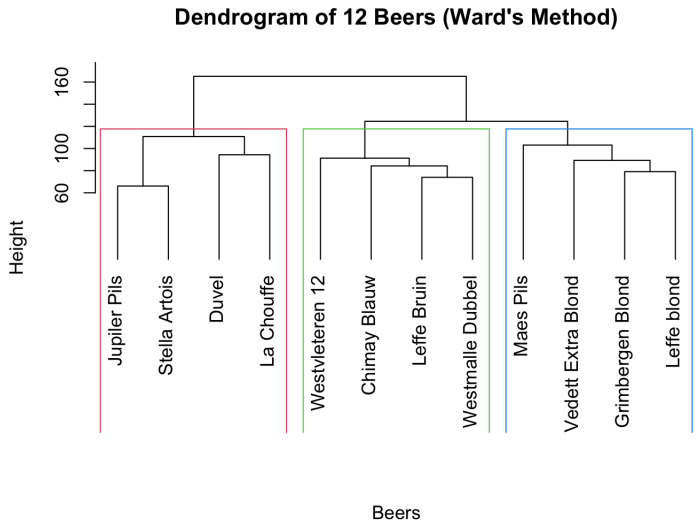

Figure 3: Dendogram and relative 3 clusters for the 12 beers obtained through Ward's method

Question B

Data Preparation

```
train_indices <- seq(1, nrow(beer), by = 2)
beer_train <- beer[train_indices, ]
beer_valid <- beer[-train_indices, ]
```

We split the data, assigning the odd observations to the train set, whereas the even ones go into the validation set. Since R does not have a built-in function to detect which centroid an observation is closest to, we built our own function to accomplish that. The following code shows how it is built:

```
clusters <- function(x, centers) {
    # compute squared euclidean distance from each sample to each cluster center
    tmp <- sapply(seq_len(nrow(x)),
                  function(i) apply(centers, 1,
                                    function(v) sum((x[i, ] - v)^2)))
    max.col(-t(tmp)) # find index of min distance
}
```

Hierarchical Clustering

```
dist_train_1 <- dist(beer_train, method = "euclidean")
hc_train_1 <- hclust(dist_train_1, method = "ward.D2")
train_groups_1 <- cutree(hc_train_1, k = num_cluster_1)
centroids_hc_1 <- as.matrix(aggregate(beer_train, by = list(train_groups_1),
                                      FUN = mean)[, -1])
valid_pred_hc_1 <- clusters(beer_valid, centroids_hc_1)
dist_valid_1 <- dist(beer_valid, method = "euclidean")
hc_valid_1 <- hclust(dist_valid_1, method = "ward.D2")
valid_independent_hc_1 <- cutree(hc_valid_1, k = num_cluster_1)
```

The first method we apply is Ward's method with Euclidean distances and $k = 3$ clusters. We use the train data to compute the first set of centroids. In particular, we specify where we want to cut the dendrogram, obtaining the different labels. We then aggregate the data using the labels we just computed, and apply the `mean` function to find the centroids for each cluster. Then, we assign each observation from the validation set to a certain cluster, depending on which of those centroids is the closest. After that, we cluster the validation data independently using the same method. In this way, each observation from the validation is associated with two different labels. We repeat this procedure with $k = 4$ clusters.

k-means clustering

```
set.seed(7)
kmeans_train_1 <- kmeans(beer_train, centers = num_cluster_1, nstart = 100)
centroids_kmeans_1 <- kmeans_train_1$centers
```

We apply the same procedure using k-means clustering on the train and validation data. For both $k = 3, 4$ we extract 100 random starting points (specified above by `nstart`). Here we do not have to calculate the centroids by hand, since the `kmeans` function already does that job.

Hierarchical clustering followed by k-means

```
start_centers_train_1 <- as.matrix(aggregate(beer_train, by =
                            list(train_groups_1), FUN = mean)[, -1])
hc_kmeans_train_1 <- kmeans(beer_train, centers = start_centers_train_1,
                            iter.max = 100)
```

Here, too, we apply the same procedure to the train and validation data. It differs slightly from what we did above, as we instruct the k-means algorithm to start its search using the centroids we have already identified in the Hierarchical Method. Once computed, we specify them inside the function using the argument `centers`. We repeat the same procedure for both $k = 3$ and $k = 4$.

Comparison between different methods using ARI

To determine which method yields the most stable solution, we compute the ARI for each label pair from those 6 models (3 methods and 2 cluster sizes). The results are displayed in Table 3. We can see that the ARI index, which measures the concordance that the same method gives to different observations, is the highest for the k-means model with 3 clusters.

Method	K	ARI_Stability
K-Means	3	0.874
K-Means	4	0.517
Hierarchical + K-Means	4	0.350
Hierarchical + K-Means	3	0.309
Hierarchical	4	0.217
Hierarchical	3	0.205

Table 3: ARI Index for different models and cluster sizes

```
set.seed(21)
best_k <- 3
final_kmeans <- kmeans(beer, centers = best_k, nstart = 100)
final_clusters <- final_kmeans$cluster
final_centers <- final_kmeans$centers
print(round(final_centers, 2))
```

We now apply the chosen model to the whole dataset. In Table 4, we find the new centroids. To characterize the three identified clusters, we analyze the coordinates of their respective centroids across the 12-dimensional space of beer rankings. Since the data is based on rankings, lower coordinate values indicate higher preference (a rank of 1 is the top choice). By comparing the centroid values for each beer, we can identify which cluster prefers a specific brand. For example, Stella Artois shows its lowest coordinate in Cluster 2 (2.93), suggesting that this group represents the core fans of this brand. Conversely, Leffe Blond displays nearly identical coordinates across all three clusters $(6.23, 6.27, 6.15)$, indicating a 'transversal' beer that lacks a specific cluster-based identity and is ranked consistently by different types of consumers.

Cluster	Jupiler	Leffe Bruin	Maes	Chimay	Grimbergen	Leffe Blond
1	7.16	6.64	10.27	4.60	6.44	6.23
2	2.19	8.50	3.65	9.28	8.03	6.27
3	3.23	9.23	9.00	7.56	7.32	6.15
Cluster	**Westvleteren**	**Westmalle**	**Duvel**	**Stella**	**La Chouffe**	**Vedett**
1	4.51	5.53	4.92	8.35	5.44	7.90
2	8.24	8.87	5.62	2.93	7.88	6.55
3	8.78	8.96	3.58	3.67	4.28	6.24

Table 4: Centroids coordinates on each beer using a k-means model with 3 clusters

Question C

Methodology and Model Fit

We now performed a two-dimensional ordinal row-conditional unfolding. The unfolding model was specified with row-conditionality to account for individual differences in ranking scales. The `unfolding()` function was utilized with the following parameters: `type = "ordinal"` and `conditionality = "row"`.

The Stress-1 value obtained for the configuration is above 0.20 (0.34), indicating a poor fit. Given that the data involves subjective human preferences and rankings, mapping such high-dimensional psychological distances into a restricted 2D space often results in higher Stress.

Results Interpretation and Visualization

In Figure 4, the distance between a person (represented by small dots) or a cluster centroid (large solid circles) and a beer (black diamonds) represents how much that person likes (prefers) a certain beer(shorter distances indicate higher preference). From the configuration plot, we can conclude the following:

1. **Cluster-Specific Preferences**: Each cluster centroid is strategically located near a specific profile of beers.
    - **Cluster 1 (Red)**: Shows a clear preference for Trappist and strong Belgian ales, such as *Chimay Blauw*, *Westvleteren 12*, and *Westmalle Dubbel*.
    - **Cluster 2 (Blue)**: Is positioned closest to *Stella Artois* and *Jupiler Pils*, indicating a preference for traditional pilsners and more commercial, accessible lagers.
    - **Cluster 3 (Green)**: Occupies a central-top position, showing an "intermediate" or specialized preference for Belgian blondes and strong ales like *Duvel*, *La Chouffe*, and *Vedett Extra Blond*.
2. **Centroid Analysis**: The centroids act as the "Ideal Points" for the typical consumer within each segment. Dimension 1 appears to separate *Pilsners/Lagers* (on the left) from *Strong Trappist/Dark Ales* (on the right). Dimension 2 seems to distinguish between *Specialty Blondes* (top) and *Dubbel/Bruin styles* (bottom), particularly evident in the distance between *Grimbergen Blond* and *Leffe Bruin*.
3. **Preference Homogeneity and Overlap**: The size of the 90% confidence ellipses reveals the consistency of tastes.
    - Cluster 2 is the most compact, suggesting very high agreement among its members regarding their preference for pilsners.
    - Cluster 1 is the largest and most dispersed, indicating a more heterogeneous group where preferences vary across different strong ales.
    - There is significant overlap between Clusters 2 and 3, suggesting that many consumers who like pilsners also appreciate blonde.

The relatively high Stress-1 value suggests that a two-dimensional space is not entirely sufficient to capture the complexity of the original ranking data without significant distortion. Consequently, while the alignment between consumer segments and specific beer characteristics is suggestive, it remains partially constrained by the model's fit. The configuration offers a useful exploratory framework for understanding preference structures, but it may not fully account for all the nuances in the individuals' ranking behaviors.

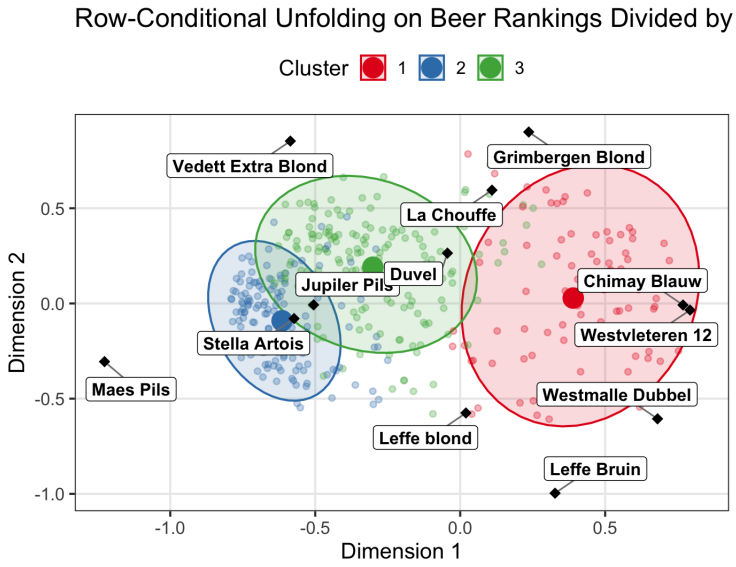

Figure 4: Row conditional unfolding of beers divided by clusters